d75 - Google Trends and Data Visualisation (start)

January 28, 2026

1 Introduction

Google Trends gives us an estimate of search volume. Let's explore if search popularity relates to other kinds of data. Perhaps there are patterns in Google's search volume and the price of Bitcoin or a hot stock like Tesla. Perhaps search volume for the term "Unemployment Benefits" can tell us something about the actual unemployment rate?

Data Sources:

Unemployment Rate from FRED

Google Trends

Yahoo Finance for Tesla Stock Price

Yahoo Finance for Bitcoin Stock Price

2 Import Statements

[108]:
```
import pandas as pd
import matplotlib.pyplot as plt
```

3 Read the Data

Download and add the .csv files to the same folder as your notebook.

[109]:
```
df_tesla = pd.read_csv('TESLA Search Trend vs Price.csv')

df_btc_search = pd.read_csv('Bitcoin Search Trend.csv')
df_btc_price = pd.read_csv('Daily Bitcoin Price.csv')

df_unemployment = pd.read_csv('UE Benefits Search vs UE Rate 2004-19.csv')
```

4 Data Exploration

4.0.1 Tesla

Challenge:

What are the shapes of the dataframes?

How many rows and columns?

What are the column names?

Complete the f-string to show the largest/smallest number in the search data column

Try the .describe() function to see some useful descriptive statistics

What is the periodicity of the time series data (daily, weekly, monthly)?

What does a value of 100 in the Google Trend search popularity actually mean?

[110]:
```
df_tesla
# df_tesla.shape
```

[110]:

	MONTH	TSLA_WEB_SEARCH	TSLA_USD_CLOSE
0	2010-06-01	3	4.766000
1	2010-07-01	3	3.988000
2	2010-08-01	2	3.896000
3	2010-09-01	2	4.082000
4	2010-10-01	2	4.368000
..	…	…	…
119	2020-05-01	16	167.000000
120	2020-06-01	17	215.962006
121	2020-07-01	24	286.152008
122	2020-08-01	23	498.320007
123	2020-09-01	31	407.339996

[124 rows x 3 columns]

[111]:
```
df_tesla.isna().values.any() #checking for number of NaN values
```

[111]: False

[112]:
```
df_tesla.describe()
```

[112]:

	TSLA_WEB_SEARCH	TSLA_USD_CLOSE
count	124.000000	124.000000
mean	8.725806	50.962145
std	5.870332	65.908389
min	2.000000	3.896000
25%	3.750000	7.352500
50%	8.000000	44.653000
75%	12.000000	58.991999
max	31.000000	498.320007

[113]:
```
max_tsl = df_tesla.TSLA_WEB_SEARCH.max()
max_tsl
```

[113]: 31

[114]:
```
min_tsl = df_tesla.TSLA_WEB_SEARCH.min()
min_tsl
```

[114]:
```
2
```

[115]:
```
print(f'Largest value for Tesla in Web Search: {max_tsl}')
print(f'Smallest value for Tesla in Web Search: {min_tsl}')
```

```
Largest value for Tesla in Web Search: 31
Smallest value for Tesla in Web Search: 2
```

4.0.2 Unemployment Data

[116]:
```
df_unemployment
# df_unemployment.tail()
# df_unemployment.describe()
# df_unemployment.shape
# df_unemployment.columns
#df_unemployment.isna().values.any()
```

[116]:

	MONTH	UE_BENEFITS_WEB_SEARCH	UNRATE
0	2004-01	34	5.7
1	2004-02	33	5.6
2	2004-03	25	5.8
3	2004-04	29	5.6
4	2004-05	23	5.6
..	…	…	…
176	2018-09	14	3.7
177	2018-10	15	3.8
178	2018-11	16	3.7
179	2018-12	17	3.9
180	2019-01	21	4.0

[181 rows x 3 columns]

[117]:
```
df_unemployment['UE_BENEFITS_WEB_SEARCH'].max()
```

[117]:
```
100
```

[118]:
```
max_un = df_unemployment['UE_BENEFITS_WEB_SEARCH'].max()
```

[119]:
```
print('Largest value for "Unemployemnt Benefits" '
      f'in Web Search: {max_un} ')
```

```
Largest value for "Unemployemnt Benefits" in Web Search: 100
```

4.0.3 Bitcoin

[120]:
```
df_btc_price
```

[120]:

	DATE	CLOSE	VOLUME
0	2014-09-17	457.334015	2.105680e+07
1	2014-09-18	424.440002	3.448320e+07
2	2014-09-19	394.795990	3.791970e+07
3	2014-09-20	408.903992	3.686360e+07
4	2014-09-21	398.821014	2.658010e+07
…	…	…	…
2199	2020-09-24	10745.548828	2.301754e+10
2200	2020-09-25	10702.290039	2.123255e+10
2201	2020-09-26	10754.437500	1.810501e+10
2202	2020-09-27	10774.426758	1.801688e+10
2203	2020-09-28	10912.536133	2.122653e+10

[2204 rows x 3 columns]

[121]:
```
df_btc_price.shape
```

[121]: (2204, 3)

[122]:
```
df_btc_search
```

[122]:

	MONTH	BTC_NEWS_SEARCH
0	2014-09	5
1	2014-10	4
2	2014-11	4
3	2014-12	4
4	2015-01	5
..	…	…
68	2020-05	22
69	2020-06	13
70	2020-07	14
71	2020-08	16
72	2020-09	13

[73 rows x 2 columns]

[123]:
```
df_btc_search.shape
```

[123]: (73, 2)

[124]:
```
max_btc = df_btc_search['BTC_NEWS_SEARCH'].max()
max_btc
```

[124]: 100

[125]:
```
print(f'largest BTC News Search: {max_btc}')
```

```
largest BTC News Search: 100
```

5 Data Cleaning

5.0.1 Check for Missing Values

Challenge: Are there any missing values in any of the dataframes? If so, which row/rows have missing values? How many missing values are there?

[126]:
```
df_tesla.isna().values.any()
```

[126]:
```
False
```

[127]:
```
df_btc_price.isna().values.any()  #vlepw oti exw missing values
```

[127]:
```
True
```

[128]:
```
df_btc_price.isna().sum()   #vlepw se pia stili exw missing values kai posa
```

[128]:
```
DATE      0
CLOSE     1
VOLUME    1
dtype: int64
```

[129]:
```
df_btc_price[df_btc_price.CLOSE.isna()]    #emfanizw ta missing values gia na␣
↪exw kai to index id wste na ta svisw
```

[129]:
```
            DATE  CLOSE  VOLUME
2148  2020-08-04    NaN     NaN
```

[130]:
```
df_btc_search.isna().values.any()
```

[130]:
```
False
```

[131]:
```
df_unemployment.isna().values.any()
```

[131]:
```
False
```

[132]:
```
print(f'Missing values for Tesla?: 0')
print(f'Missing values for U/E?: 0')
print(f'Missing values for BTC Search?: 0')
```

```
Missing values for Tesla?: 0
Missing values for U/E?: 0
Missing values for BTC Search?: 0
```

[133]:
```
print(f'Missing values for BTC price?: {df_btc_price.isna().values.any()}')
```

```
Missing values for BTC price?: True
```

[134]:
```
print(f'Number of missing values: {df_btc_price.isna().sum().sum()}')
```

```
Number of missing values: 2
```

Challenge: Remove any missing values that you found.

[135]:
```
df_btc_price.dropna(inplace = True)     #Diwxnw ta NaN values antika8istwntas to␣
↪arxiko df
```

5.0.2 Convert Strings to DateTime Objects

Challenge: Check the data type of the entries in the DataFrame MONTH or DATE columns. Convert any strings in to Datetime objects. Do this for all 4 DataFrames. Double check if your type conversion was successful.

[136]:
```
print(df_tesla['MONTH'][1]) #checking
type(df_tesla['MONTH'][1])
```

```
2010-07-01
```

[136]:
```
str
```

[137]:
```
print(pd.to_datetime(df_tesla['MONTH'][1])) #checking after pd.to_datetime
type(pd.to_datetime(df_tesla['MONTH'][1]))
```

```
2010-07-01 00:00:00
```

[137]:
```
pandas._libs.tslibs.timestamps.Timestamp
```

[138]:
```
df_tesla.MONTH = pd.to_datetime(df_tesla['MONTH'])  #converting the whole column
```

[139]:
```
df_tesla.MONTH.head()
```

[139]:
```
0   2010-06-01
1   2010-07-01
2   2010-08-01
3   2010-09-01
4   2010-10-01
Name: MONTH, dtype: datetime64[ns]
```

5.0.3 Converting from Daily to Monthly Data

Pandas .resample() documentation

[140]:
```
df_btc_price.DATE = pd.to_datetime(df_btc_price['DATE'])
```

[141]:
```
df_btc_price.DATE.head()
```

[141]:
```
0   2014-09-17
1   2014-09-18
2   2014-09-19
3   2014-09-20
4   2014-09-21
Name: DATE, dtype: datetime64[ns]
```

[142]:
```
df_btc_monthly = df_btc_price.resample('M', on='DATE').last()
```

[143]:
```
df_btc_monthly
```

[143]:

DATE	CLOSE	VOLUME
2014-09-30	386.944000	3.470730e+07
2014-10-31	338.321014	1.254540e+07
2014-11-30	378.046997	9.194440e+06
2014-12-31	320.192993	1.394290e+07
2015-01-31	217.464005	2.334820e+07
…	…	…
2020-05-31	9461.058594	2.777329e+10
2020-06-30	9137.993164	1.573580e+10
2020-07-31	11323.466797	2.316047e+10
2020-08-31	11680.820313	2.228593e+10
2020-09-30	10912.536133	2.122653e+10

[73 rows x 2 columns]

6 Data Visualisation

6.0.1 Notebook Formatting & Style Helpers

[144]:
```
# Create locators for ticks on the time axis
```

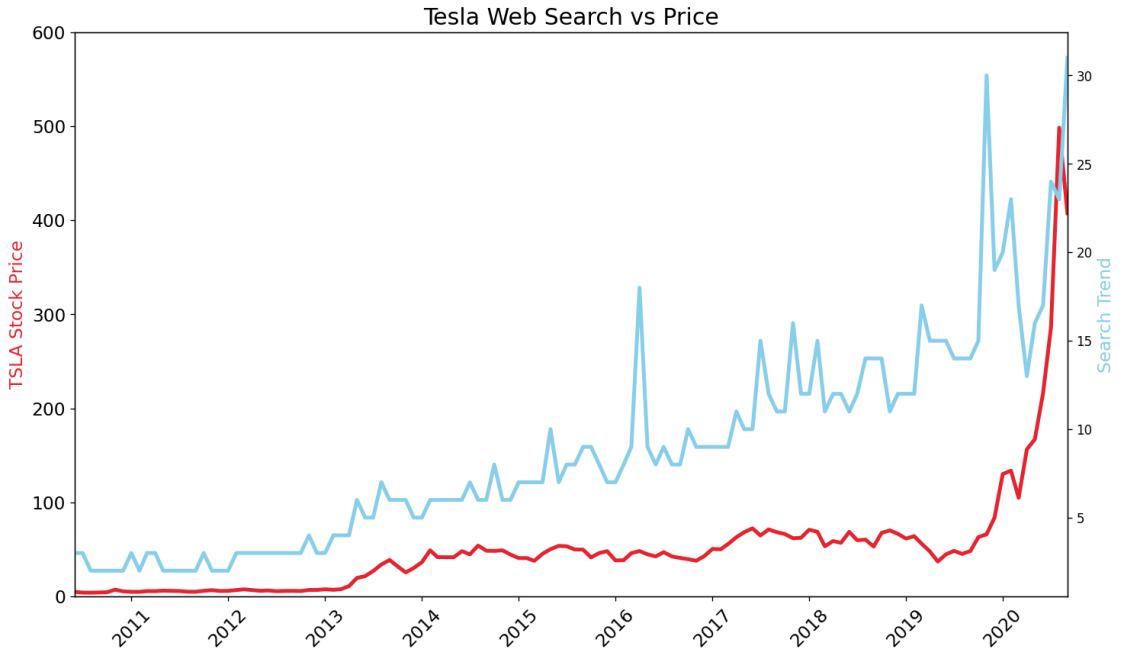
[145]:
```
# Register date converters to avoid warning messages
```

6.0.2 Tesla Stock Price v.s. Search Volume

Challenge: Plot the Tesla stock price against the Tesla search volume using a line chart and two different axes. Label one axis 'TSLA Stock Price' and the other 'Search Trend'.

[146]:
```
plt.figure(figsize=(14,8), dpi = 120)
plt.title('Tesla Web Search vs Price', fontsize = 18)
plt.xticks(fontsize=14, rotation=45)
plt.yticks(fontsize=14)
ax1 = plt.gca()
ax2 = ax1.twinx()

ax1.set_ylabel('TSLA Stock Price', color='#E6232E', fontsize=14)
```

```
ax2.set_ylabel('Search Trend', color='skyblue', fontsize=14)

ax1.set_ylim([0,600])
ax1.set_xlim([df_tesla.MONTH.min(), df_tesla.MONTH.max()])

ax1.plot(df_tesla['MONTH'], df_tesla['TSLA_USD_CLOSE'], color='#E6232E',␣
 ↪linewidth = 3)
ax2.plot(df_tesla['MONTH'], df_tesla['TSLA_WEB_SEARCH'], color='skyblue',␣
 ↪linewidth = 3)

plt.show()
```



Challenge: Add colours to style the chart. This will help differentiate the two lines and the axis labels. Try using one of the blue colour names for the search volume and a HEX code for a red colour for the stock price. Hint: you can colour both the axis labels and the lines on the chart using keyword arguments (kwargs).

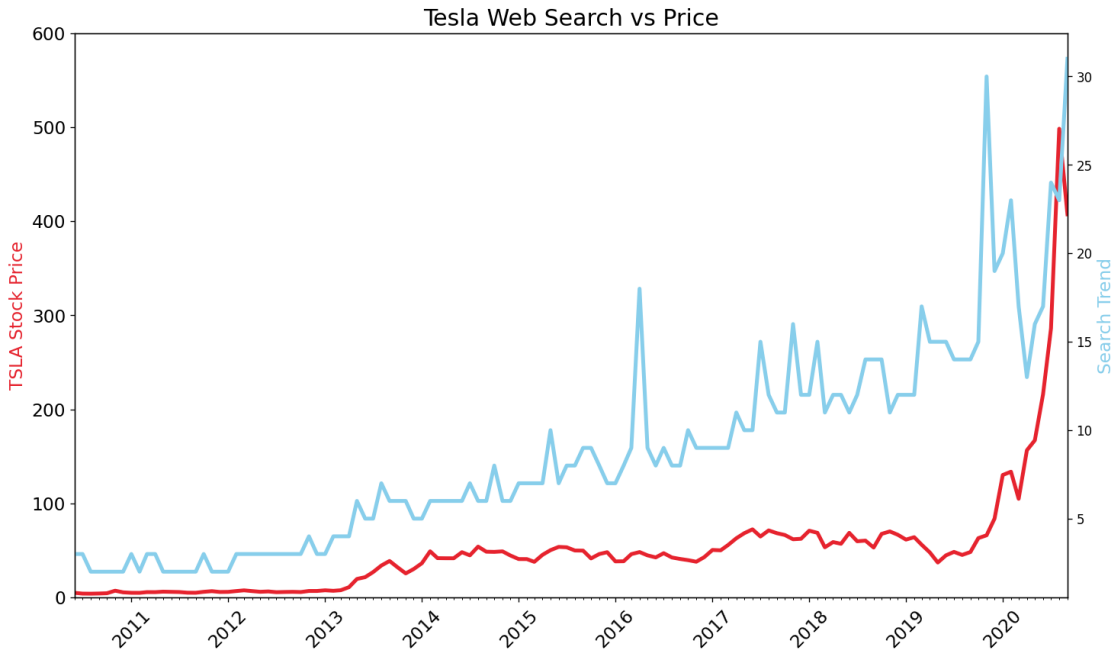
[147]:
```
#done above
```

Challenge: Make the chart larger and easier to read. 1. Increase the figure size (e.g., to 14 by 8). 2. Increase the font sizes for the labels and the ticks on the x-axis to 14. 3. Rotate the text on the x-axis by 45 degrees. 4. Make the lines on the chart thicker. 5. Add a title that reads 'Tesla Web Search vs Price' 6. Keep the chart looking sharp by changing the dots-per-inch or DPI value. 7. Set minimum and maximum values for the y and x axis. Hint: check out methods like set_xlim(). 8. Finally use plt.show() to display the chart below the cell instead of relying on the automatic notebook output.

[148]:
```
#done above
```

How to add tick formatting for dates on the x-axis.

[149]:
```
import matplotlib.dates as mdates

#code from earlier
plt.figure(figsize=(14,8), dpi = 120)
plt.title('Tesla Web Search vs Price', fontsize = 18)
plt.xticks(fontsize=14, rotation=45)
plt.yticks(fontsize=14)
ax1 = plt.gca()
ax2 = ax1.twinx()

ax1.set_ylabel('TSLA Stock Price', color='#E6232E', fontsize=14)
ax2.set_ylabel('Search Trend', color='skyblue', fontsize=14)

ax1.set_ylim([0,600])
ax1.set_xlim([df_tesla.MONTH.min(), df_tesla.MONTH.max()])

ax1.plot(df_tesla['MONTH'], df_tesla['TSLA_USD_CLOSE'], color='#E6232E',␣
 ↪linewidth = 3)
ax2.plot(df_tesla['MONTH'], df_tesla['TSLA_WEB_SEARCH'], color='skyblue',␣
 ↪linewidth = 3)

#create locators for ticks on the time axis
years = mdates.YearLocator()
months = mdates.MonthLocator()
years_fmt = mdates.DateFormatter('%Y')

#format the ticks
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)
ax1.xaxis.set_minor_locator(months)


plt.show()
```

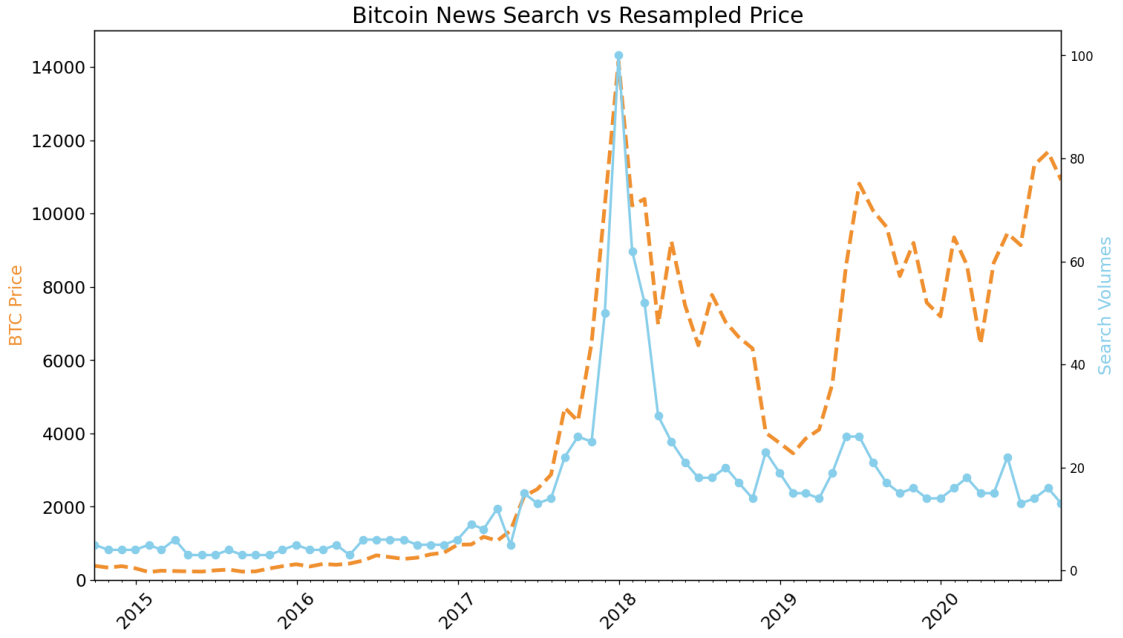


6.0.3 Bitcoin (BTC) Price v.s. Search Volume

Challenge: Create the same chart for the Bitcoin Prices vs. Search volumes. 1. Modify the chart title to read 'Bitcoin News Search vs Resampled Price' 2. Change the y-axis label to 'BTC Price' 3. Change the y- and x-axis limits to improve the appearance 4. Investigate the linestyles to make the BTC price a dashed line 5. Investigate the marker types to make the search datapoints little circles 6. Were big increases in searches for Bitcoin accompanied by big increases in the price?

[150]:
```
#code from earlier
plt.figure(figsize=(14,8), dpi = 120)
plt.title('Bitcoin News Search vs Resampled Price', fontsize = 18)
plt.xticks(fontsize=14, rotation=45)
plt.yticks(fontsize=14)
ax1 = plt.gca()
ax2 = ax1.twinx()

ax1.set_ylabel('BTC Price', color='#F08F2E', fontsize=14)
ax2.set_ylabel('Search Volumes', color='skyblue', fontsize=14)

ax1.set_ylim([0,15000])
ax1.set_xlim([df_btc_monthly.index.min(), df_btc_monthly.index.max()])

ax1.plot(df_btc_monthly.index, df_btc_monthly['CLOSE'], color='#F08F2E',␣
 ↪linewidth = 3, linestyle = '--')
ax2.plot(df_btc_monthly.index, df_btc_search['BTC_NEWS_SEARCH'],␣
 ↪color='skyblue', linewidth = 2, marker = 'o')
```

```
#create locators for ticks on the time axis
years = mdates.YearLocator()
months = mdates.MonthLocator()
years_fmt = mdates.DateFormatter('%Y')

#format the ticks
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)
ax1.xaxis.set_minor_locator(months)


plt.show()
```



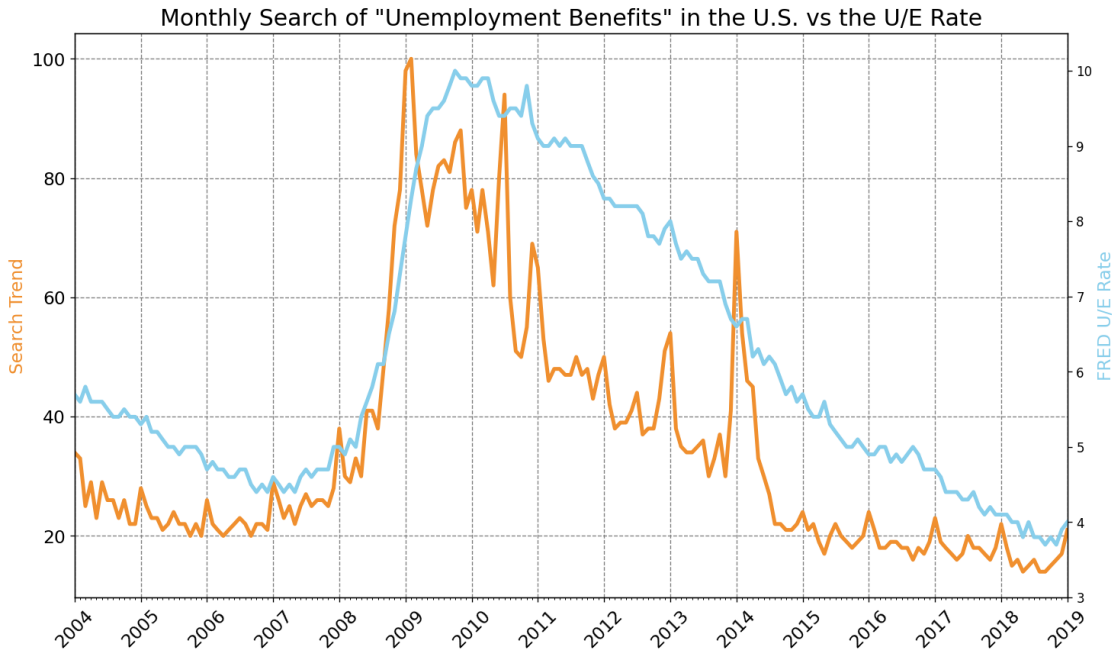
[ ]:

6.0.4 Unemployement Benefits Search vs. Actual Unemployment in the U.S.

Challenge Plot the search for "unemployment benefits" against the unemployment rate. 1. Change the title to: Monthly Search of "Unemployment Benefits" in the U.S. vs the U/E Rate 2. Change the y-axis label to: FRED U/E Rate 3. Change the axis limits 4. Add a grey grid to the chart to better see the years and the U/E rate values. Use dashes for the line style 5. Can you discern any seasonality in the searches? Is there a pattern?

[156]:

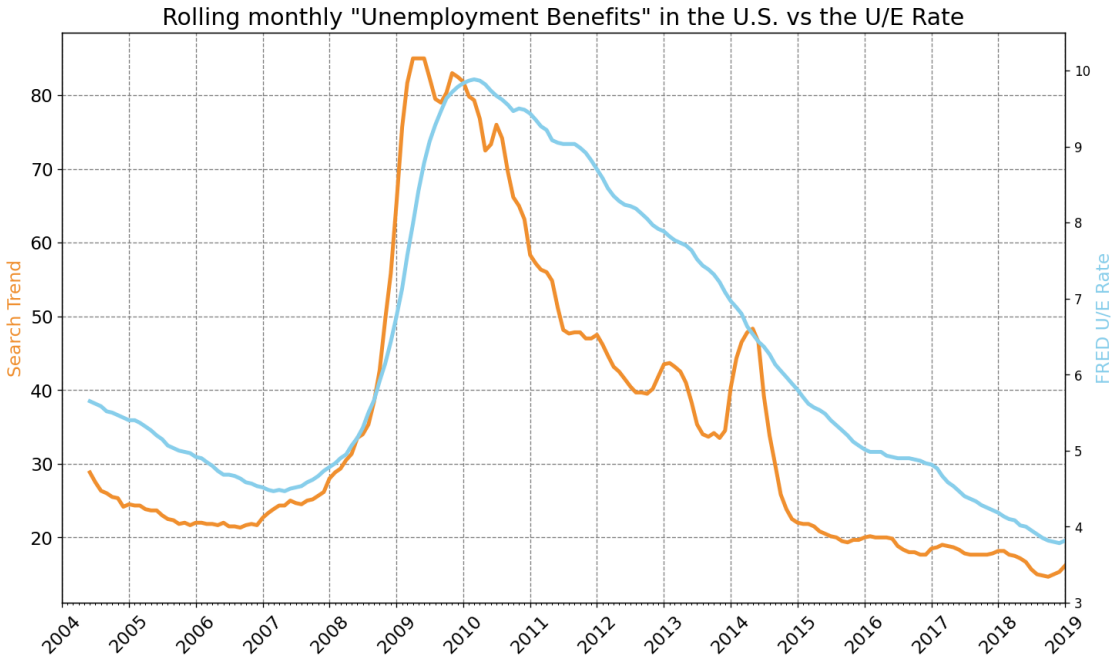
```
df_unemployment['MONTH'] = pd.to_datetime(df_unemployment['MONTH'])
#If i didn't do this line ,i would get an error due to the fact that you're␣
 ↪passing a sequence
#(in this case, a list) to ax1.set_xlim(), which expects individual values for␣
 ↪the left and right limits, not a sequence.


#code from earlier
plt.figure(figsize=(14,8), dpi = 120)
plt.title('Monthly Search of "Unemployment Benefits" in the U.S. vs the U/E␣
 ↪Rate', fontsize = 18)
plt.xticks(fontsize=14, rotation=45)
plt.yticks(fontsize=14)
ax1 = plt.gca()
ax2 = ax1.twinx()
ax1.grid(color='grey', linestyle = '--')

ax1.set_ylabel('Search Trend', color='#F08F2E', fontsize=14)
ax2.set_ylabel('FRED U/E Rate', color='skyblue', fontsize=14)

ax2.set_ylim([3,10.5])
ax1.set_xlim([df_unemployment.MONTH.min(), df_unemployment.MONTH.max()])

ax1.plot(df_unemployment['MONTH'], df_unemployment['UE_BENEFITS_WEB_SEARCH'] ,␣
 ↪color='#F08F2E', linewidth = 3)
ax2.plot(df_unemployment['MONTH'], df_unemployment['UNRATE'], color='skyblue',␣
 ↪linewidth = 3)

#format the ticks
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)
ax1.xaxis.set_minor_locator(months)


plt.show()
```



Challenge: Calculate the 3-month or 6-month rolling average for the web searches. Plot the 6-month rolling average search data against the actual unemployment. What do you see in the chart? Which line moves first?

[169]:
```
plt.figure(figsize=(14,8), dpi = 120)
plt.title('Rolling monthly "Unemployment Benefits" in the U.S. vs the U/E␣
 ↪Rate', fontsize = 18)
plt.xticks(fontsize=14, rotation=45)
plt.yticks(fontsize=14)
ax1 = plt.gca()
ax2 = ax1.twinx()
ax1.grid(color='grey', linestyle = '--')

ax1.set_ylabel('Search Trend', color='#F08F2E', fontsize=14)
ax2.set_ylabel('FRED U/E Rate', color='skyblue', fontsize=14)

ax2.set_ylim([3,10.5])
ax1.set_xlim([df_unemployment.MONTH.min(), df_unemployment.MONTH.max()])

# Calculate the rolling average over a 6 month window
roll_df = df_unemployment[['UE_BENEFITS_WEB_SEARCH', 'UNRATE']].
 ↪rolling(window=6).mean()

ax2.plot(df_unemployment.MONTH, roll_df.UNRATE, 'skyblue', linewidth=3)
ax1.plot(df_unemployment.MONTH, roll_df.UE_BENEFITS_WEB_SEARCH,␣
 ↪color='#F08F2E', linewidth=3)
```

```
#format the ticks
ax1.xaxis.set_major_locator(years)
ax1.xaxis.set_major_formatter(years_fmt)
ax1.xaxis.set_minor_locator(months)


plt.show()
```



6.0.5 Including 2020 in Unemployment Charts

Challenge: Read the data in the 'UE Benefits Search vs UE Rate 2004-20.csv' into a DataFrame. Convert the MONTH column to Pandas Datetime objects and then plot the chart. What do you see?

[173]:
```
df_ue_2020 = pd.read_csv("UE Benefits Search vs UE Rate 2004-20.csv")
df_ue_2020
```

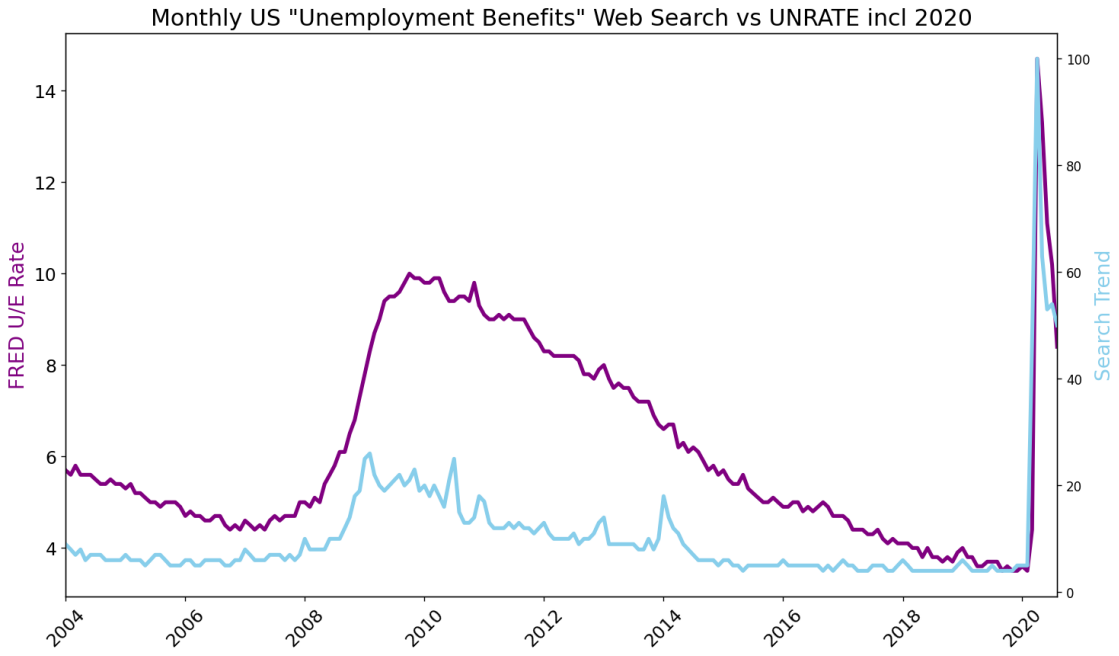
[173]:

	MONTH	UE_BENEFITS_WEB_SEARCH	UNRATE
0	2004-01	9	5.7
1	2004-02	8	5.6
2	2004-03	7	5.8
3	2004-04	8	5.6
4	2004-05	6	5.6
..	…	…	…
195	2020-04	100	14.7

```
196  2020-05                        63    13.3
197  2020-06                        53    11.1
198  2020-07                        54    10.2
199  2020-08                        50     8.4

[200 rows x 3 columns]
```

[174]:
```
df_ue_2020.MONTH = pd.to_datetime(df_ue_2020.MONTH)
```

[175]:
```
plt.figure(figsize=(14,8), dpi=120)
plt.yticks(fontsize=14)
plt.xticks(fontsize=14, rotation=45)
plt.title('Monthly US "Unemployment Benefits" Web Search vs UNRATE incl 2020',␣
 ↪fontsize=18)

ax1 = plt.gca()
ax2 = ax1.twinx()

ax1.set_ylabel('FRED U/E Rate', color='purple', fontsize=16)
ax2.set_ylabel('Search Trend', color='skyblue', fontsize=16)

ax1.set_xlim([df_ue_2020.MONTH.min(), df_ue_2020.MONTH.max()])

ax1.plot(df_ue_2020.MONTH, df_ue_2020.UNRATE, 'purple', linewidth=3)
ax2.plot(df_ue_2020.MONTH, df_ue_2020.UE_BENEFITS_WEB_SEARCH, 'skyblue',␣
 ↪linewidth=3)

plt.show()
```



[ ]: